

Se codificará cada respuesta en JavaScript y donde sea necesario se incluirá como código entre etiquetas `<script></script>` paraa renderizar efecto en página web con visualización posible en consola, o bien directamente se codificará aparte para ser ejecutado por consola.

CURSO: DESARROLLO DE APLICACIONES CON TECNOLOGIAS WEB		Calificación
Expediente:		
Módulo: MF0491_3. Unidad Formativa: UF1842		
Examen Intermedio 2 de la Unidad Formativa: UF1842		

Nombre: VICTOR VISÚS GARCÍA	Fecha: 25/03/2026
-----------------------------	-------------------

En el HTML, modifiko la posición de de la etiqueta SCRIPT con `src=main.js`, para que funcione correctamente

1. Dado el siguiente código HTML

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <title>Page Title</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <script src="main.js"></script>
  </head>
  <body>
    <input type="radio" name="eleccion" value="opcion1" checked>Opcion1<br/>
    <input type="radio" name="eleccion" value="opcion2">Opcion2<br/>
    <input type="radio" name="eleccion" value="opcion3">Opcion3<br/>
  </body>
</html>
```

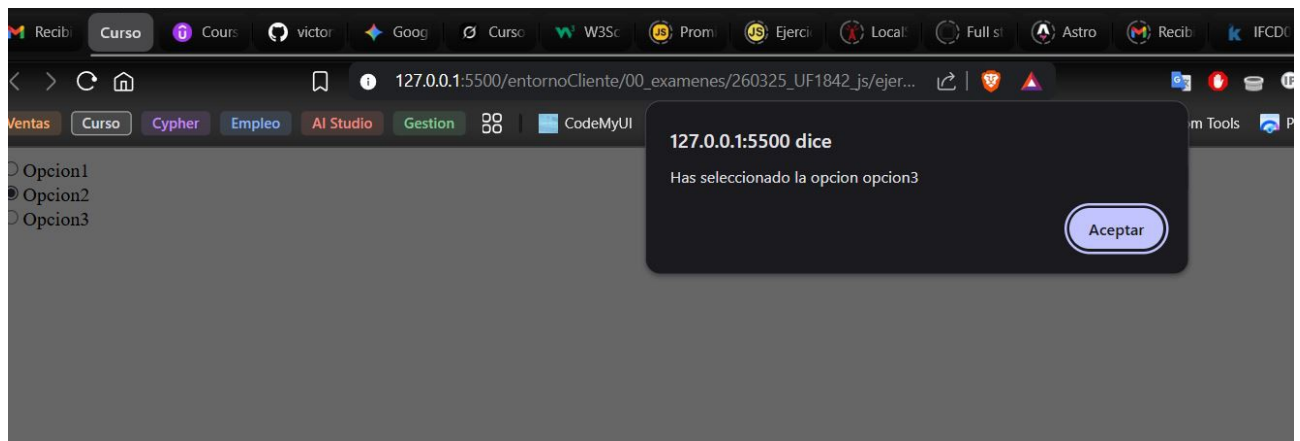
Crear una función en Javascript tal que cuando uno cualquiera de los tres radio-buttons se selecciona se muestre un mensaje, tipo alert, diciendo que opción se seleccionó.

Indicación: hacer uso de las funciones tipo `getElementsByXXX(...)` o `querySelectorXXX(...)`, que se consideren adecuadas así como de la adecuada estructura de control en bucle para gestionar la tarea pedida. El evento desencadenante y el nombre de su acción pueden implementarse en cada elemento o bien en un script separado.

(2 puntos)

```
const radios = document.querySelectorAll('input[type="radio"]');

for (let i = 0; i < radios.length; i++) {
  radios[i].addEventListener('change', () => {
    alert(`Has seleccionado la opcion ${radios[i].value}`);
  });
}
```



2. Añadir entre `<body>` y `</body>` un elemento `<input>` de tipo texto (`type="text"`) con valor inicial a cadena vacía (`""`) de modo que al pasar el ratón (no al hacer click) sobre el contenido de cualquiera de los tres radio-buttons anteriores actualice el contenido de ese input con el contenido textual de dicho radio-button (Opcion1, etc). Indicación: implementar el método `addEventListener(...)` que captura la escucha y desencadena la acción.
(1,5 puntos)

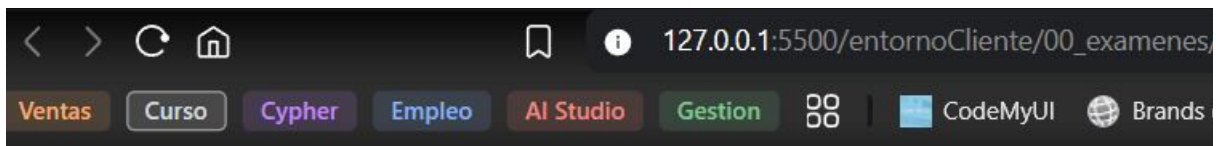
– Al HTML añadido:

```
<!-- AGREGO EL CÓDIGO INDICADO EN EL ENUNCIADO, del ejercicio 2-->
<div role="group">
  <label for="valor-opcion-id">Valor seleccionado</label>
  <input type="text" name="valor-opcion" id="valor-opcion-id" disabled>
</div>
```

– El script anterior, con las modificaciones solicitadas en este ejercicio, queda:

```
const opcion = document.querySelectorAll('input[type="radio"]');
const cajaTexto = document.querySelector('input[name="valor-opcion"]');

for (let i = 0; i < opcion.length; i++) {
  opcion[i].addEventListener('change', () => {
    alert(`Has seleccionado la opcion ${opcion[i].value}`);
  });
  opcion[i].addEventListener('mouseenter', () => {
    cajaTexto.value = opcion[i].value;
  });
}
```



- ☒ Opcion1
☐ Opcion2
☐ Opcion3

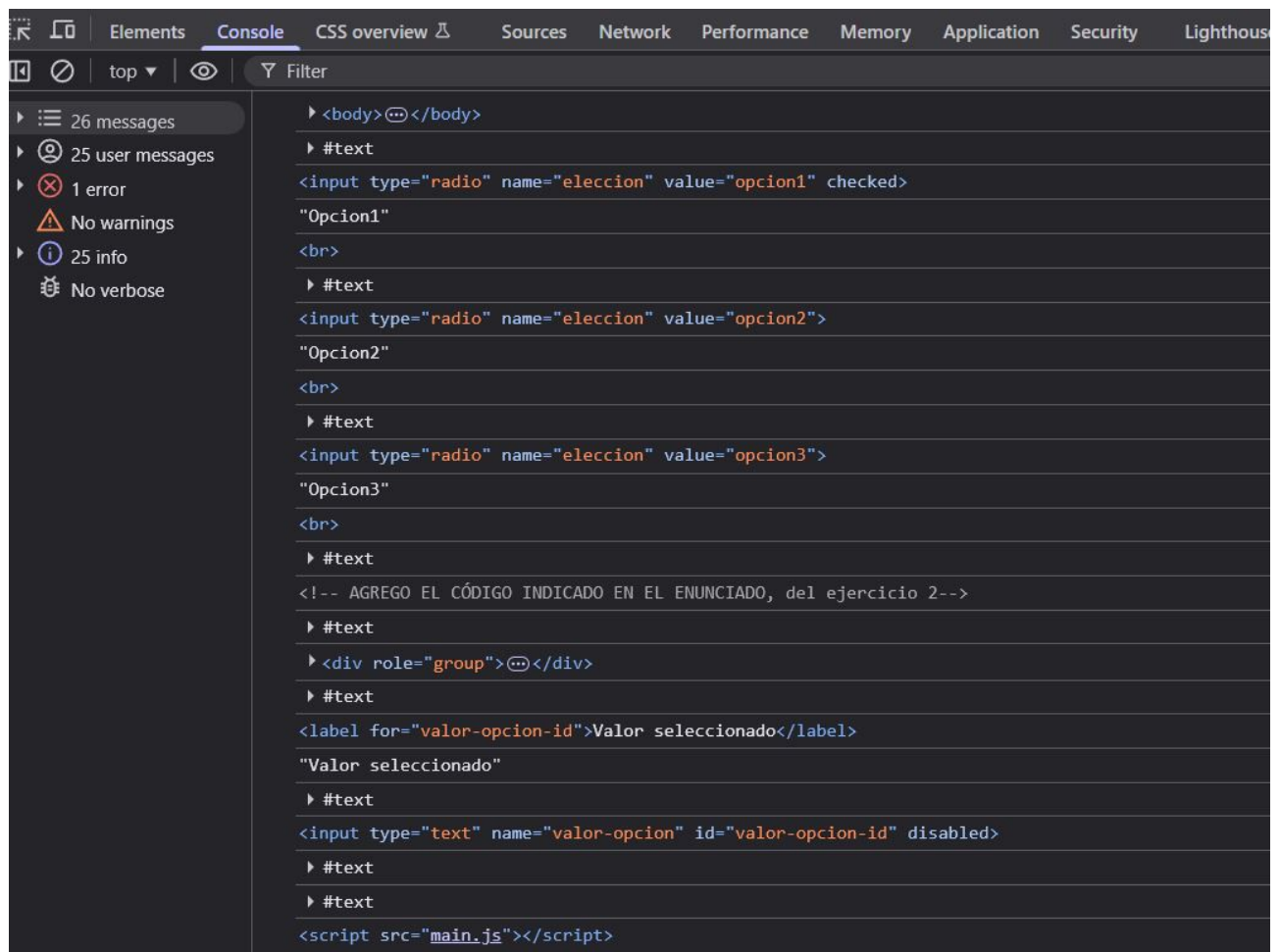
Valor seleccionado

opcion3

3. Muestra a través de mensajes de consola la totalidad de los Nodos (no sólo los elementos, sino también comentarios y textos) contenidos en el código html del ejercicio 2 antes y después de haber ejecutado el código Javascript del mismo ejercicio.

(1,5 puntos)

```
function escanearNodos(nodoPadre) {  
  console.log(nodoPadre);  
  for (let i = 0; i < nodoPadre.childNodes.length; i++) {  
    escanearNodos(nodoPadre.childNodes[i]);  
  }  
}  
  
const nodoPadre = document.body;  
escanearNodos(nodoPadre);
```



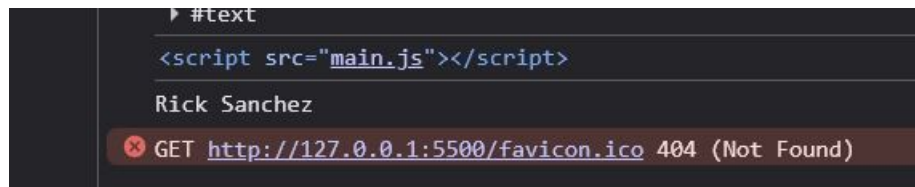
4. Completar esta función asíncrona para devolver el nombre (name) del personaje de la serie: “Rick and Morty”:

```
---- function obtenerPersonaje() {  
  try {  
    const ---- = ---- ----('https://rickandmortyapi.com/api/character/1');  
    const data = ---- response.----;  
    console.log(data.----);  
  } catch (error) {  
    console.----('Error: Personaje no encontrado', ----);  
  }  
}
```

```
async function obtenerPersonaje() {  
  try {  
    const response = await fetch('https://rickandmortyapi.com/api/character/1');  
    const data = await response.json();  
    console.log(data.name);  
  } catch (error) {  
    console.error('Error: Personaje no encontrado', error);  
  }  
}  
  
obtenerPersonaje();
```

Y probarla a continuación:

(2,5 puntos)



5. Crear la clase Personaje que se basa en la anterior API y se compone de los atributos: nombre (el nombre del personaje) y url. Modificar la plantilla del ejercicio 4 para que sea un método de dicha clase capaz de crear o actualizar un objeto con el nombre del personaje (primer nivel del API) y la url del primer episodio.

(2,5 puntos)

```
class Personaje {  
  constructor(name = null, url = null) {  
    this.name = name;  
    this.url = url;  
  }  
  
  async obtenerPersonaje() {  
    try {  
      const response = await fetch(this.url);  
      const data = await response.json();  
  
      this.name = data.name;
```

```
    this.url = data.episode[0];  
  } catch (error) {  
    console.error('Error: Personaje no encontrado', error);  
  }  
}  
}
```